

벨만 방정식과 최적성

무한한 미래를 유한한 연립방정식으로 바꾸는 재귀 구조, 그리고 $\max$ 가 들어올 때 달라지는 것

sajacaros
sajacall82@naver.com

2026년 8월 25일

초록

앞 장에서 마르코프 결정 과정의 목표를 상태 가치 함수 $v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$ 로 정의하였으나, 이를 정의대로 계산하려면 무한히 이어지는 궤적을 모두 펼쳐야 하고 최적 정책을 찾으려면 $|\mathcal{A}|^{|\mathcal{S}|}$ 개의 정책을 모두 평가해야 했다. 벨만 방정식(Bellman equation)은 이 두 벽을 동시에 허문다. 수익의 재귀 구조 $G_t = R_t + \gamma G_{t+1}$ 한 줄에서 출발하여, 무한한 미래를 “한 걸음의 보상 + 이웃 상태의 가치”로 접어 넣으면 가치 함수는 미지수 $|\mathcal{S}|$ 개짜리 연립일차방정식의 해가 된다. 본고는 이 도출을 백업 다이어그램의 확률 셈법에서부터 따라간 뒤, 행동 가치 함수 $q_\pi(s, a)$ 를 경유하여 $\sum_a \pi(a | s)$ 를 $\max_a$ 로 바꾼 벨만 최적 방정식에 이르는 경로를 정리한다. 원서[1] 3장은 수식 중심이어서 실행 가능한 코드를 제공하지 않으므로, 두 칸짜리 그리드 월드와 3×4 그리드 월드를 소재로 예제 네 편을 구성하고 여기에 네 가지 정량 분석을 더하였다. 첫째, 반복 대입의 오차 감소 비율이 γ 이하가 아니라 정확히 γ 로 수렴함을 P_π 의 스펙트럼으로 설명하고, 첫 걸음에서만 비율이 0.818로 어긋나는 이유를 고윳값 분해로 밝힌다. 둘째, 직접 해법 $O(|\mathcal{S}|^3)$ 과 반복법 $O(K|\mathcal{S}||\mathcal{A}|)$ 의 손익분기가 $|\mathcal{S}| \approx \sqrt{3K|\mathcal{A}|}$ 이며 $\gamma = 0.9$, $\varepsilon = 10^{-10}$ 에서 약 51개 상태임을 보인다. 셋째, 무작위 정책의 Q 를 한 번 탐욕화하면 3×4 그리드의 10개 상태 전부에서 가치가 오르지만 (평균 $-0.212 \rightarrow 0.760$) 세 상태는 여전히 최적(0.826)에 못 미침을 보여, “한 번의 탐욕화”와 “반복”을 가르는 간극을 수치로 제시한다. 넷째, 벨만 최적 방정식의 손풀이에서 네 가지 결정적 정책 가정 중 하나만이 자기일관임을 전수 확인하고, $\max$ 연산자의 비확장성으로부터 최적 벨만 연산자 역시 γ -축소 사상임을 증명한다. 본고의 절 구성은 원서 3장의 목차를 그대로 따르되, 각 절 끝에 “[보충]”으로 표시한 분석을 덧붙였다.

주제어: 강화 학습, 벨만 방정식, 벨만 최적 방정식, 행동 가치 함수, 축소 사상, 고정점, 동적 프로그래밍

차례

1	서론	2
1.1	전수 탐색의 벽	2
1.2	본고의 구성과 기여	2
2	벨만 방정식 도출	3
2.1	확률과 기댓값(사전 준비)	3
2.2	벨만 방정식 도출	5
2.2.1	수익의 재귀 구조	5
2.2.2	두 층의 확률: 정책과 상태 전이	5
2.2.3	벨만 방정식	6
3	벨만 방정식의 예	6
3.1	두 칸짜리 그리드 월드	6
3.1.1	행렬 형태와 정확해	8

3.1.2	3 × 4 그리드 월드로	9
3.2	벨만 방정식의 의미	9
3.3	[보충] 반복 대입의 수렴률은 정확히 γ 다	10
3.4	[보충] 직접 해법과 반복법의 비용	11
4	행동 가치 함수(Q 함수)와 벨만 방정식	13
4.1	행동 가치 함수	13
4.2	행동 가치 함수를 이용한 벨만 방정식	14
4.3	[보충] 한 번의 탐욕화가 주는 것과 주지 못하는 것	14
5	벨만 최적 방정식	15
5.1	상태 가치 함수의 벨만 최적 방정식	15
5.2	행동 가치 함수의 벨만 최적 방정식	16
5.3	[보충] 최적 벨만 연산자도 축소 사상이다	17
6	벨만 최적 방정식의 예	18
6.1	벨만 최적 방정식 적용	18
6.2	최적 정책 구하기	19
6.3	[보충] 네 가지 가정 중 무엇이 자기일관인가	19
6.4	세 방법의 대조	20
7	정리	20
A	재현 정보	21

1 서론

1.1 전수 탐색의 벽

앞 장[2]에서 마르코프 결정 과정(MDP)을 정식화하고 목표를 상태 가치 함수

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s], \quad G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \quad (1)$$

로 정의하였다. 그리고 두 칸짜리 그리드 월드에서 최적 정책을 찾을 때는 결정적 정책 네 가지를 모두 나열하여 각각의 가치를 500걸음 시뮬레이션으로 재는 전수 탐색을 썼다.

이 방법은 두 군데에서 막힌다. 하나는 정책 하나의 가치를 재는 일이 끝나지 않는다는 것이다. 식 (1)의 G_t 는 무한급수이므로 정의대로는 끝까지 더할 수 없고, 500걸음에서 자르는 것은 어디까지나 근사다. 다른 하나는 재야 할 정책의 수다. 결정적 정책의 수는 $|A|^{|S|}$ 이므로 상태가 100개이고 행동이 4가지면 $4^{100} \approx 1.6 \times 10^{60}$ 가지를 세어야 한다. 두 칸 세계에서 네 가지를 세어 본 것은 문제가 작았기 때문이지 방법이 좋았기 때문이 아니다.

그림 1이 첫 번째 벽을 그대로 보여준다. 앞 장에서 그린 백업 다이어그램인데, 맨 아래에서 가지가 점선으로 계속 이어진다. 수익을 정의대로 구하려면 이 점선을 끝까지 따라가야 한다.

벨만 방정식의 착상은 단순하다. 점선 아래에 있는 것도 결국 “그 상태에서 시작하는 수익의 기댓값”이므로 $v_{\pi}(s')$ 자체다. 그렇다면 무한한 가치를 펼치는 대신 한 층만 내려간 뒤 나머지를 $v_{\pi}(s')$ 로 바꿔 쓰면 된다. 그 대가로 v_{π} 가 등식의 양변에 나타나지만, 그것은 무한급수가 아니라 유한한 연립방정식이다.

1.2 본고의 구성과 기여

본고는 원서 3장의 목차를 그대로 따른다. §2에서 백업 다이어그램의 확률 셈법을 정리한 뒤 수익의 재귀 구조로부터 벨만 방정식을 도출하고, §3에서 그 식을 두 칸 세계와 3 × 4 그리드에 적용하여

그림 3-1 결정적 백업 다이어그램(왼쪽)과 확률적 백업 다이어그램(오른쪽)

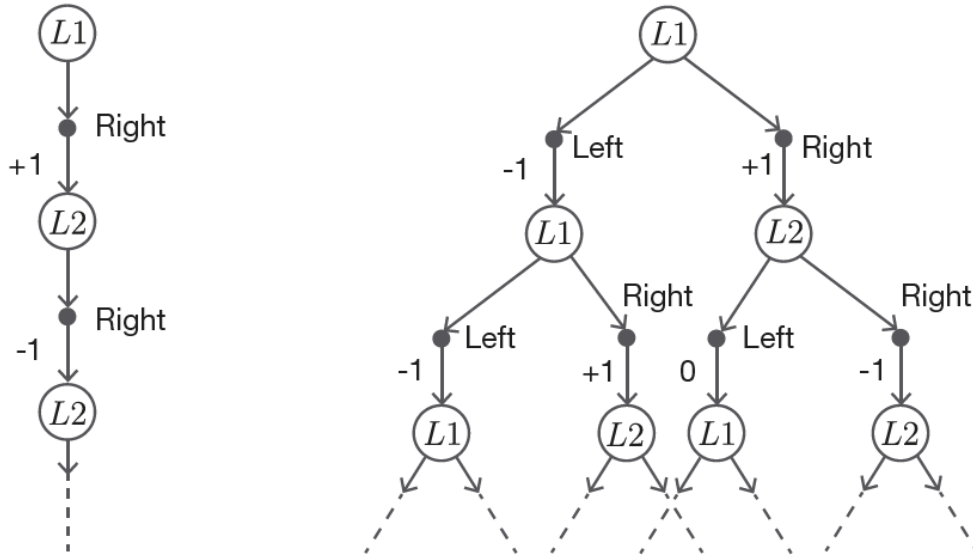


그림 1. 결정적 정책(왼쪽)과 확률적 정책(오른쪽)의 백업 다이어그램. 맨 아래에서 가치가 점선으로 계속 이어지는 것이 문제다. 벨만 방정식의 발상은 한 단계만 내려가고 그 아래 전체를 가치 함수 하나로 대신하는 것이다. 출처: 원서 그림 3-1.

연립일차방정식으로 푼다. §4에서 행동 가치 함수를 도입하여 V 와 Q 의 상호 변환을 정리하고, §5에서 $\sum_a \pi(a | s)$ 를 $\max_a$ 로 바꾼 벨만 최적 방정식을 세운 뒤 §6에서 그것을 다시 두 칸 세계에 적용한다. §7이 정리다.

원서 3장에는 소스 코드가 없다. 본고가 추가한 것은 예제 네 편과 다음 네 가지 분석이며, 각각을 해당 절 끝에 “[보충]”으로 표시하였다.

1. **반복 대입의 수렴률은 정확히 γ 다**(§3.3). 축소 사상 논증이 주는 것은 $\|e_{k+1}\| \leq \gamma \|e_k\|$ 라는 부등식뿐이지만, 실측 비율은 0.9000...로 등식이다. P_π 가 행 확률 행렬이라 스펙트럼 반지름이 1이라는 사실에서 이 등식이 나오며, 첫 걸음에서만 비율이 0.818로 어긋나는 것은 초기 오차에 섞인 $\lambda = 0$ 성분이 한 번에 소멸하기 때문임을 고깃값 분해로 보인다.
2. **직접 해법과 반복법의 손익분기**(§3.4). 정확해를 주는 직접 해법이 있는데도 4장이 반복법으로 가는 이유를 연산량과 기억 용량으로 정량화한다. 손익분기는 $|S| \approx \sqrt{3K|\mathcal{A}|}$ 이며 $\gamma = 0.9$, $\varepsilon = 10^{-10}$ 에서 약 51개 상태다.
3. **한 번의 탐욕화가 주는 것과 주지 못하는 것**(§4.3). 무작위 정책의 Q 를 한 번 탐욕화하면 3×4 그리드의 10개 상태 전부에서 가치가 오르지만(평균 $-0.212 \rightarrow 0.760$), 아래쪽 세 칸은 최적값에 최대 0.279 못 미친다. 정책 반복법이 왜 “반복”인지가 이 간극이다.
4. **최적 연산자의 축소성과 손풀이의 경우 분석**(§5.3, §6.3). $\max$ 의 비확장성으로부터 최적 벨만 연산자가 γ -축소 사상임을 증명하고, 네 가지 결정적 정책 가정 중 자기일관인 것이 하나뿐임을 전수 확인한다. 예제가 정확히 333회에서 멈추는 이유도 부동소수점 분해능으로 설명한다.

2 벨만 방정식 도출

2.1 확률과 기댓값(사전 준비)

벨만 방정식은 결국 기댓값 계산이므로, 먼저 백업 다이어그램에서 기댓값을 읽는 연습을 해 둔다. 가장 단순한 예는 주사위 한 번 던지기다.

그림 3-2 주사위의 백업 다이어그램

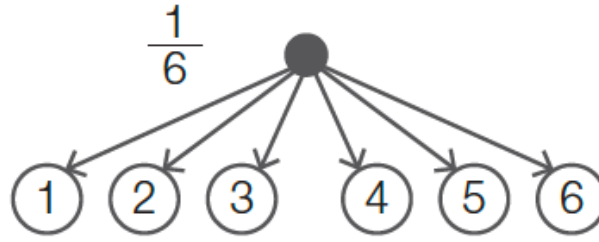


그림 2. 주사위 한 번 던지기의 백업 다이어그램. 검은 점에서 여섯 갈래로 뻗고 가지마다 확률 $1/6$ 이 붙는다. 출처: 원서 그림 3-2.

가지에 붙은 확률과 끝의 값을 곱해 모두 더한 것이 기댓값이다.

$$\mathbb{E}[X] = \sum_x x p(x) = \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = 3.5. \quad (2)$$

다음은 두 단계짜리 문제다. 주사위를 던져 홀수가 나오면 공정한 동전을, 짝수가 나오면 앞면이 잘 나오는 동전을 던진다. 동전이 앞면이면 주사위 눈만큼 보상을 받고 뒷면이면 0이다.

그림 3-3 주사위와 동전을 순서대로 던지는 문제

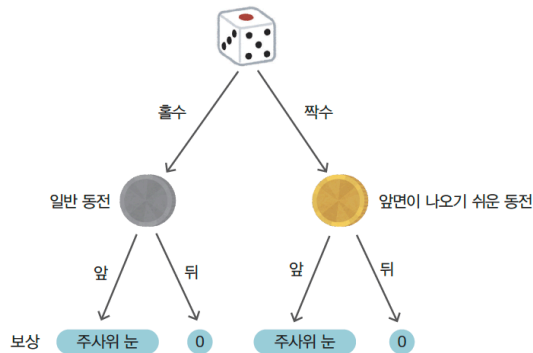


그림 3-4 주사위와 동전을 순서대로 던지는 문제의 백업 다이어그램

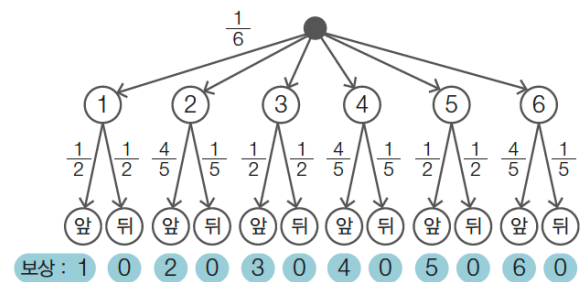


그림 3. 두 단계짜리 문제(왼쪽)와 이를 하나의 백업 다이어그램으로 편 것(오른쪽). 두 번째 단계의 확률이 첫 번째 단계의 결과에 의존한다는 점이 MDP와 같은 구조다. 홀수 가지의 앞면 확률은 $1/2$, 짝수 가지는 $4/5$ 이다. 출처: 원서 그림 3-3, 3-4.

이때 기댓값은 맨 위에서 맨 아래까지 가지의 확률을 모두 곱하고 거기에 마지막 값을 곱해 더한 것이다.

$$\begin{aligned} \mathbb{E}[R] &= \frac{1}{6} \left(1 \cdot \frac{1}{2} + 2 \cdot \frac{4}{5} + 3 \cdot \frac{1}{2} + 4 \cdot \frac{4}{5} + 5 \cdot \frac{1}{2} + 6 \cdot \frac{4}{5} \right) \\ &= \frac{1}{6} (0.5 + 1.6 + 1.5 + 3.2 + 2.5 + 4.8) = \frac{14.1}{6} = 2.35. \end{aligned} \quad (3)$$

뒷면 가지는 보상이 0이라 합에서 사라지므로 12개 가지 중 6개 항만 남았다.

식 (3)에서 확인할 것은 두 가지다. 첫째, 가지를 따라 확률을 곱해 더한다는 셈법이 층이 늘어도 그대로 통한다. 둘째, 두 번째 층의 확률이 첫 번째 층의 결과에 따라 달라진다. 이 두 성질이 그대로 벨만 방정식의 유도에 쓰인다. MDP에서는 첫 번째 층이 정책이고 두 번째 층이 상태 전이일 뿐이다.

2.2 벨만 방정식 도출

2.2.1 수익의 재귀 구조

출발점은 수익의 정의다.

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots \quad (4)$$

한 시각 뒤의 수익도 같은 모양이다.

$$G_{t+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \quad (5)$$

식 (4)에서 첫 항을 떼고 나머지를 γ 로 묶으면 식 (5)이 그대로 나타난다.

$$G_t = R_t + \gamma G_{t+1}. \quad (6)$$

이 한 줄에서 이 장의 모든 식이 나온다. 식 (6)이 하는 일은 무한히 이어지는 합을 한 걸음과 나머지 전부로 나눈 것뿐이지만, 나머지 전부가 원래 문제와 같은 모양이라는 점이 결정적이다. 앞에서 이 식은 보상 열을 뒤에서부터 접어 올리는 계산 요령으로만 쓰였는데, 이제 그것이 방정식이 된다.

이제 식 (6)을 가치 함수의 정의

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \quad (7)$$

에 대입하고 기댓값의 선형성으로 두 항을 나눈다.

$$v_\pi(s) = \mathbb{E}_\pi[R_t \mid S_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_t = s]. \quad (8)$$

그림 3-6 전개 중인 식

$$v_\pi(s) = \mathbb{E}_\pi[R_t \mid S_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_t = s]$$

첫 번째 항 전개 다음은 이쪽 항

$$\sum_{a, s'} \pi(a \mid s) p(s' \mid s, a) r(s, a, s')$$

그림 4. 식 (8)의 두 항. 하늘색으로 칠해진 두 덩어리를 각각 전개한다. 왼쪽은 한 걸음의 기댓값이므로 §2.1의 샘플법을 그대로 적용하면 되고, 오른쪽은 그 뒤에 처리한다. 출처: 원서 그림 3-6.

2.2.2 두 층의 확률: 정책과 상태 전이

식 (8)의 두 항을 전개하려면 어떤 확률이 걸려 있는지 알아야 한다. 그림 5이 그것을 보여준다.

§2.1의 두 단계 문제와 구조가 같다. 맨 위에서 s' 까지 내려가는 한 가지의 확률은 두 층의 확률을 곱한 $\pi(a \mid s) p(s' \mid s, a)$ 이고, 그 끝에 붙은 값이 보상 $r(s, a, s')$ 이다. 따라서 식 (8)의 첫 항은

$$\mathbb{E}_\pi[R_t \mid S_t = s] = \sum_a \sum_{s'} \pi(a \mid s) p(s' \mid s, a) r(s, a, s') \quad (9)$$

이 된다. 둘째 항도 같다. 상태가 s' 으로 바뀌고 나면 그 뒤의 기대 수익은 정의상 $v_\pi(s')$ 이므로

$$\mathbb{E}_\pi[G_{t+1} \mid S_t = s] = \sum_a \sum_{s'} \pi(a \mid s) p(s' \mid s, a) v_\pi(s') \quad (10)$$

이다. 여기가 유도의 핵심이다. G_{t+1} 은 그 자체로 무한급수인데, 그 무한급수의 기댓값을 다시 펼치지 않고 $v_\pi(s')$ 이라는 이름으로 대신했다. 그림 1의 점선이 여기서 잘린다.

그림 3-5 상태와 행동의 관계

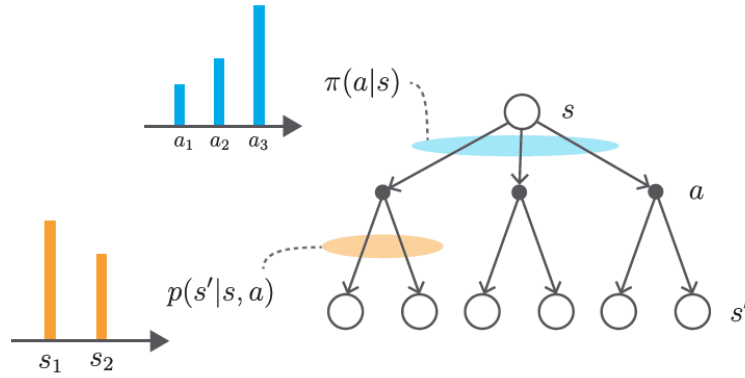


그림 5. 한 걸음에 걸린 두 개의 확률분포. 하늘색(상태 → 행동)은 정책 $\pi(a | s)$ 이고, 주황색(행동 → 다음 상태)은 상태 전이 확률 $p(s' | s, a)$ 다. 왼쪽의 막대그래프 두 개는 각각이 확률분포임을 강조한 것이다. 출처: 원서 그림 3-5.

2.2.3 벨만 방정식

식 (9)과 (10)을 식 (8)에 넣고 공통의 $\sum$ 으로 묶으면 벨만 방정식을 얻는다.

$$v_{\pi}(s) = \sum_a \sum_{s'} \pi(a | s) p(s' | s, a) \{ r(s, a, s') + \gamma v_{\pi}(s') \}. \quad (11)$$

이중 합을 안팎으로 분리해 쓰면 코드로 옮기기 좋은 형태가 된다.

$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s'} p(s' | s, a) \{ r(s, a, s') + \gamma v_{\pi}(s') \}. \quad (12)$$

바깥쪽 $\sum_a$ 는 정책에 대한 평균이고 안쪽 $\sum_{s'}$ 은 상태 전이에 대한 평균이다. 상태 전이가 결정적이어서 $s' = f(s, a)$ 로 정해지면 안쪽 합이 사라진다.

$$v_{\pi}(s) = \sum_a \pi(a | s) \{ r(s, a, f(s, a)) + \gamma v_{\pi}(f(s, a)) \}. \quad (13)$$

본고가 다루는 두 환경은 모두 상태 전이가 결정적이므로 실제 계산에는 식 (13)을 쓴다.

식 (11)이 말하는 바는 한 문장으로 줄어든다.

어떤 상태의 가치는 (한 걸음 보상의 기댓값) + $\gamma \times$ (이웃 상태 가치의 기댓값)이다.

무한한 미래를 다 볼 필요가 없어졌다는 것이 얻은 것이고, v_{π} 가 좌변과 우변에 동시에 나타나는 자기참조 구조가 치른 대가다. 다음 절에서 보듯 그 대가는 연립방정식을 푸는 정도의 값이다.

3 벨만 방정식의 예

3.1 두 칸짜리 그리드 월드

앞 장에서 쓴 두 칸짜리 그리드 월드를 그대로 가져온다. 이번에 평가할 정책은 L1과 L2 어디에서나 Left와 Right를 0.5씩 고르는 무작위 정책이다.

환경의 규칙은 앞 장과 같다. 벽에 부딪히면 -1 , L2로 이동하면 사과를 먹어 $+1$, L2에서 L1로 가면 0이며 $\gamma = 0.9$ 다. 이제 상태마다 벨만 방정식을 한 줄씩 세운다.

그림 3-8 넓게 퍼져나가는 백업 다이어그램(이번 문제에서 상태는 결정적으로 전이됨)

그림 3-7 두 칸짜리 그리드 월드(벽에 부딪히면 -1, 사과를 얻으면 +1, 사과는 계속 생성)

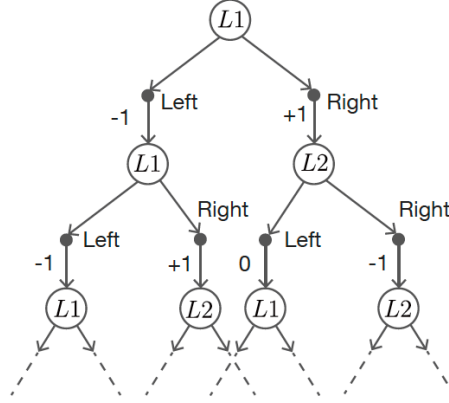
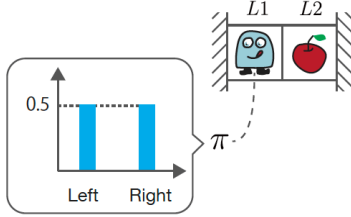


그림 6. 두 칸짜리 그리드 월드와 무작위 정책(왼쪽), 그 백업 다이어그램(오른쪽). 왼쪽 막대그래프 두 개의 높이가 같은 것이 $\pi(a|s) = 0.5$ 이다. 오른쪽에서 상태에서 나오는 가지는 둘로 갈라지지만 행동에서 나오는 화살표는 하나뿐인데, 이것이 상태 전이가 결정적이라는 뜻이며 따라서 식 (13)을 쓸 수 있다. 출처: 원서 그림 3-7, 3-8.

그림 3-9 $v_\pi(L1)$ 을 구하기 위한 백업 다이어그램

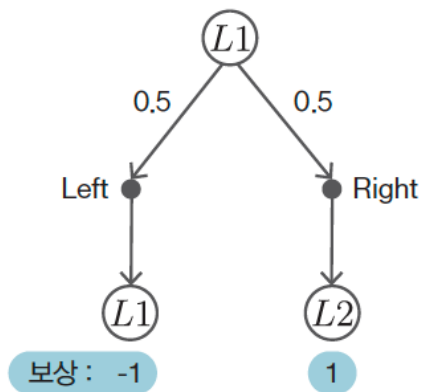


그림 3-10 $v_\pi(L2)$ 를 구하기 위한 백업 다이어그램

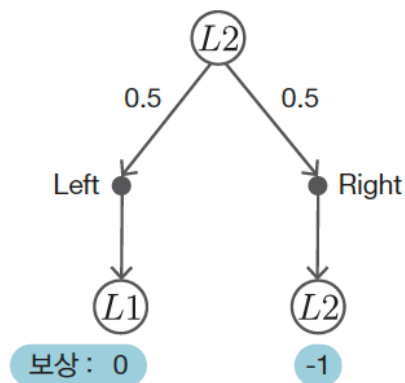


그림 7. $v_\pi(L1)$ (왼쪽)과 $v_\pi(L2)$ (오른쪽)에 대한 한 단계짜리 다이어그램. 그림 6 오른쪽의 맨 윗부분만 떼어낸 것으로, 아래가 점선으로 이어지지 않고 L1, L2에서 끊겨 있다. 그 아래 전체를 $v_\pi(L1)$, $v_\pi(L2)$ 로 대신하는 것이 벨만 방정식의 요령이다. 출처: 원서 그림 3-9, 3-10.

그림 7을 그대로 읽으면 다음 두 식이 나온다.

$$v_{\pi}(L1) = 0.5\{-1 + 0.9 v_{\pi}(L1)\} + 0.5\{1 + 0.9 v_{\pi}(L2)\}, \quad (14)$$

$$v_{\pi}(L2) = 0.5\{0 + 0.9 v_{\pi}(L1)\} + 0.5\{-1 + 0.9 v_{\pi}(L2)\}. \quad (15)$$

정리하면 미지수 두 개짜리 연립일차방정식이다.

$$\begin{cases} -0.55 v_{\pi}(L1) + 0.45 v_{\pi}(L2) = 0, \\ 0.45 v_{\pi}(L1) - 0.55 v_{\pi}(L2) = 0.5. \end{cases} \quad (16)$$

풀면 $v_{\pi}(L1) = -2.25$, $v_{\pi}(L2) = -2.75$ 다. 무작위로 걸으면 벽에 부딪히는 일이 잦아 값이 음수가 된다.

3.1.1 행렬 형태와 정확해

식 (16)을 세운 과정은 상태 수가 늘어도 달라지지 않는다. 벨만 방정식은 v_{π} 에 대한 일차식이므로 언제나 행렬로 묶인다. 정책 π 아래의 상태 전이 행렬과 기대 보상 벡터를

$$[P_{\pi}]_{ss'} = \sum_a \pi(a | s) p(s' | s, a), \quad [r_{\pi}]_s = \sum_a \pi(a | s) \sum_{s'} p(s' | s, a) r(s, a, s') \quad (17)$$

로 정의하면 식 (11)은 다음과 같이 압축된다.

$$v = r_{\pi} + \gamma P_{\pi} v \iff (I - \gamma P_{\pi}) v = r_{\pi}. \quad (18)$$

이 방정식이 언제나 유일한 해를 갖는다는 것은 다음과 같이 확인된다. P_{π} 는 각 행의 합이 1인 확률 행렬이므로 스펙트럼 반지름이 $\rho(P_{\pi}) = 1$ 이고, 따라서 $\rho(\gamma P_{\pi}) = \gamma < 1$ 이다. 그러면 $I - \gamma P_{\pi}$ 의 고윳값은 모두 0에서 떨어져 있어 가역이며, 역행렬은 노이만 급수

$$(I - \gamma P_{\pi})^{-1} = \sum_{k=0}^{\infty} (\gamma P_{\pi})^k \quad (19)$$

로 주어진다. 식 (19)의 우변은 “ k 걸음 뒤에 어느 상태에 있을 확률에 γ^k 를 곱해 모두 더한 것”이므로, 이 급수는 식 (4)의 무한급수를 행렬로 다시 쓴 것에 지나지 않는다. 즉 $\gamma < 1$ 이라는 조건은 수익을 수렴시키는 조건인 동시에 연립방정식을 풀 수 있게 하는 조건이다.

```
P, r = env.policy_matrices(pi, gamma)      # P_π와 r_π를 만든다
I = np.eye(len(P))
v = np.linalg.solve(I - gamma * P, r)      # (I - γP_π)v = r_π
```

코드 1. 연립방정식으로 푸는 벨만 방정식 (s01_bellman_linear.py)

무작위 정책에서 P_{π} 와 r_{π} 는 다음과 같다.

$$P_{\pi} = \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{pmatrix}, \quad r_{\pi} = \begin{pmatrix} 0 \\ -0.5 \end{pmatrix}. \quad (20)$$

r_{π} 의 첫 성분이 0인 것은 L1에서 벽에 부딪히는 -1과 사과를 먹는 +1이 반반이라 상쇄되기 때문이다. 표 1에 결과를 정리하였다.

표 1의 아래 행이 이 장의 요점이다. 앞 장에서 5.2632를 얻으려면 정책을 고정하고 500걸음을 시뮬레이션해야 했다. 여기서는 2×2 연립방정식을 한 번 풀었다. 근사가 정확해로 바뀌었고 걸음 수라는 임의의 절단 지점도 사라졌다.

표 1. 두 칸짜리 그리드 월드의 상태 가치($\gamma = 0.9$). 무작위 정책의 값은 식 (16)의 해와 같고, 결정적 정책 μ 의 값은 앞 장이 500걸음 시뮬레이션으로 얻었던 값과 같다.

정책	$v(L1)$	$v(L2)$	비고
무작위 $\pi(a s) = 0.5$	-2.2500	-2.7500	원서 3.2절의 값
$\mu = (\text{Right}, \text{Left})$	5.2632	4.7368	앞 장의 전수 탐색 최적 정책

표 2. 3×4 그리드 월드에서 무작위 정책($\pi(a | s) = 0.25$)의 상태 가치($\gamma = 0.9$). (0, 3)은 목표 상태(가치 0), (1, 1)은 벽, (1, 3)은 폭탄이다. 12원 연립방정식의 정확해다.

	$w = 0$	$w = 1$	$w = 2$	$w = 3$
$h = 0$	0.0257	0.0946	0.2055	0.0000
$h = 1$	-0.0318	(벽)	-0.4980	-0.3727
$h = 2$	-0.1034	-0.2210	-0.4368	-0.7857

3.1.2 3×4 그리드 월드로

같은 코드의 후반부는 3×4 그리드 월드를 12원 연립방정식으로 푼다. 이 환경은 다음 장에서 반복 계산으로 다시 풀 문제다.

표 2에서 오른쪽 아래 -0.7857이 가장 낮다. 폭탄(1, 3) 바로 아래이면서 목표에서 멀어, 무작위로 걸으면 폭탄에 빨려 들어갈 확률이 높은 자리다. 목표 상태 왼쪽의 0.2055가 가장 높은 것도 같은 이유의 뒤집힌 표현이다.

이 값들은 다음 장의 반복 정책 평가가 구하는 값과 최대 오차 4.644×10^{-10} 로 일치한다. 반복 계산에 쓴 종료 문턱이 10^{-10} 이었으므로 이는 두 방법이 같은 방정식을 서로 다른 방식으로 푼 것임을 보여준다.

3.2 벨만 방정식의 의의

벨만 방정식이 없다면 가치를 구하는 길은 두 가지뿐이고 둘 다 막혀 있다.

표 3. 벨만 방정식이 없을 때 $v_\pi(s)$ 에 이르는 두 경로. 어느 쪽도 끝나지 않는다.

방법	막히는 곳
수익의 정의대로 계산	무한히 이어지는 항을 모두 더해야 한다(앞 장은 500걸음에서 잘랐다)
모든 궤적을 나열해 기댓값	그림 6 오른쪽처럼 가지가 2^n 으로 폭발한다

벨만 방정식은 이 문제를 미지수 $|\mathcal{S}|$ 개짜리 연립일차방정식으로 바꾼다. 상태가 100개면 100원 연립방정식이며, 유한하고 컴퓨터가 곧바로 풀 수 있는 크기다. 앞 장의 전수 탐색이 마주한 4^{100} 과는 차원이 다르다. 무한급수와 지수적 나열이라는 두 벽이 동시에 사라지는 자리가 여기다.

다만 정확해를 주는 이 길에도 한계가 있다. 식 (18)을 직접 푸는 데는 $|\mathcal{S}| \times |\mathcal{S}|$ 행렬이 필요하고 연산량이 $O(|\mathcal{S}|^3)$ 이므로, 상태가 아주 많아지면 감당하기 어려워진다. 그래서 실제로는 우변에 계속 대입해 근사해로 다가가는 반복 대입을 쓰며, 그것이 다음 장의 동적 프로그래밍이다.

반복 대입이 왜 되는지는 식 (18)을 다시 보면 알 수 있다. 이 식은 우변에도 v 가 있으므로 고정점 방정식이기도 하다. 벨만 연산자

$$(\mathcal{T}_\pi v)(s) = \sum_a \pi(a | s) \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v(s')\}, \quad \mathcal{T}_\pi v = r_\pi + \gamma P_\pi v \quad (21)$$

를 두면 벨만 방정식은 $v_\pi = \mathcal{T}_\pi v_\pi$ 로 쓰인다.

$(\mathcal{T}_\pi v)(s)$ 라는 표기를 푸는 법. 여기서 v 는 숫자 하나가 아니라 상태마다 값이 하나씩 적힌 표, 곧 길이 $|S|$ 짜리 벡터다. $\mathcal{T}_\pi$ 는 그런 표를 하나 받아 같은 크기의 표를 하나 내놓는 기계이며, 이렇게 함수를 넣으면 함수가 나오는 것을 연산자(operator)라 부른다. 따라서 $\mathcal{T}_\pi v$ 자체가 새로운 표이고, 뒤에 붙은 (s) 는 그 새 표에서 s 칸의 값을 꺼내라는 뜻이다. 괄호가 $\mathcal{T}_\pi(v(s))$ 가 아니라 $(\mathcal{T}_\pi v)(s)$ 인 이유가 여기에 있다.

기계가 하는 일은 식 (21)의 우변 그대로다. 지금 손에 든 표 v 를 잠정적인 가치 추정치로 믿고, 상태 s 에서 한 걸음만 실제로 굴려 보상 $r(s, a, s')$ 을 받은 다음, 그 이후는 도착한 칸의 값 $v(s')$ 을 그대로 갖다 쓴다. 요컨대 한 걸음은 진짜로 계산하고 나머지는 지금의 추정치로 때우는 것이며, 그 결과를 모든 상태에 대해 적어 놓은 표가 $\mathcal{T}_\pi v$ 다. 참값 v_π 를 넣었다면 때운 부분도 참값이므로 나온 표 역시 v_π 일 수밖에 없는데, 그것이 곧 $v_\pi = \mathcal{T}_\pi v_\pi$ 다.

고정점 방정식이란. $x = f(x)$ 처럼 넣은 것이 그대로 다시 나오는 꼴의 방정식을 고정점 방정식이라 하고, 그것을 만족하는 x 를 f 의 고정점(fixed point)이라 한다. 익숙한 예가 $x = \cos x$ 다. 계산기에 아무 수나 넣고 $\cos$ 을 계속 누르면 0.739085...로 빨려 들어가는데, 그 값이 $\cos$ 의 고정점이다.

벨만 방정식이 정확히 이 꼴이다. 좌변에도 v_π 가 있고 우변에도 v_π 가 있으므로, 이 식을 푼다는 것은 $\mathcal{T}_\pi$ 에 넣었을 때 자기 자신이 도로 나오는 표를 찾는 일이다. 그리고 $\cos$ 의 예가 그랬듯이 그런 표는 아무 값에서 출발해 계속 대입하는 것만으로 찾을 수 있다.

$$v_{k+1} = \mathcal{T}_\pi v_k = r_\pi + \gamma P_\pi v_k \quad (22)$$

다만 아무 f 에서나 되는 것은 아니고 f 가 두 점 사이의 거리를 줄이는 함수여야 하는데($\cos$ 이 그렇다), 벨만 연산자가 그 조건을 만족한다는 것이 다음 절의 명제 1이다. 수렴이 보장되는 근거와 그 속도는 §3.3에서, 직접 해법과의 손익분기는 §3.4에서 따지며, 이 반복을 실제로 돌려 보는 예제가 s02_bellman_iterative.py다.

3.3 [보충] 반복 대입의 수렴률은 정확히 γ 다

식 (22)이 수렴하는 근거는 $\mathcal{T}_\pi$ 가 축소 사상이라는 것이다.

명제 1 (벨만 연산자의 축소성). 임의의 $v, w \in \mathbb{R}^{|S|}$ 에 대해 $\|\mathcal{T}_\pi v - \mathcal{T}_\pi w\|_\infty \leq \gamma \|v - w\|_\infty$ 이다.

증명. $\mathcal{T}_\pi v - \mathcal{T}_\pi w = \gamma P_\pi(v - w)$ 이고, P_π 의 각 행은 합이 1인 음이 아닌 수들이므로 $|(P_\pi(v - w))_s| \leq \sum_{s'} [P_\pi]_{ss'} |v(s') - w(s')| \leq \|v - w\|_\infty$ 이다. $\square$

명제 1과 바나흐 고정점 정리로부터 고정점의 존재와 유일성, 그리고 임의의 초깃값에서 출발한 식 (22)의 수렴이 따라온다. 오차 $e_k := v_k - v_\pi$ 에 대해

$$\|e_{k+1}\|_\infty \leq \gamma \|e_k\|_\infty \implies \|e_k\|_\infty \leq \gamma^k \|e_0\|_\infty \quad (23)$$

이다. 그런데 예제 s02_bellman_iterative.py가 실측한 감소 비율은 γ 이하가 아니라 0.9000과 0.5000, 즉 γ 와 정확히 같았다. 표 4가 그 실측이다.

부등식이 등식으로 나타나는 데에는 이유가 있다. 식 (22)에서 오차는

$$e_k = (\gamma P_\pi)^k e_0 = \gamma^k P_\pi^k e_0 \quad (24)$$

로 정확히 전개된다. P_π 가 행 확률 행렬이면 $P_\pi \mathbf{1} = \mathbf{1}$ 이므로 $\mathbf{1}$ 은 고윳값 1의 고유벡터이고, e_0 를 $\mathbf{1}$ 방향 성분과 나머지로 나누어 $e_0 = c\mathbf{1} + u$ 라 쓰면

$$e_k = \gamma^k (c\mathbf{1} + P_\pi^k u) \quad (25)$$

이 된다. $P_\pi^k u$ 는 나머지 고윳값들의 크기에 따라 줄어들 뿐이므로, $c \neq 0$ 인 한 $\|e_k\|_\infty$ 는 $\gamma^k |c|$ 에 접근한다. 즉 점근 수렴률은 γ 이하가 아니라 정확히 γ 이며, 이는 $\rho(P_\pi) = 1$ 이 언제나 성립하는

표 4. $v_0 = 0$ 에서 출발한 반복 대입의 오차와 감소 비율. $k \geq 1$ 에서 비율이 γ 와 소수점 아래 10자리까지 같으며, $k = 0 \rightarrow 1$ 에서만 어긋난다.

$\gamma = 0.9$			$\gamma = 0.5$		
k	$\ e_k\ _\infty$	$\ e_{k+1}\ /\ e_k\ $	k	$\ e_k\ _\infty$	$\ e_{k+1}\ /\ e_k\ $
0	2.7500000000	0.8181818182	0	0.7500000000	0.3333333333
1	2.2500000000	0.9000000000	1	0.2500000000	0.5000000000
2	2.0250000000	0.9000000000	2	0.1250000000	0.5000000000
3	1.8225000000	0.9000000000	3	0.0625000000	0.5000000000
4	1.6402500000	0.9000000000	4	0.0312500000	0.5000000000
5	1.4762250000	0.9000000000	5	0.0156250000	0.5000000000

데서 오는 일반적 성질이다. 식 (23)의 부등호를 등호로 만들 수 없는 예외는 초기 오차가 1 방향 성분을 갖지 않는($c = 0$) 특수한 경우뿐이다.

두 칸 세계에서는 이 분해가 손으로 끝난다. 식 (20)의 P_π 는 고윳값이 $\{1, 0\}$ 이고 $P_\pi^2 = P_\pi$ 인 사영 행렬이다. $v_0 = 0$ 이므로

$$e_0 = -v_\pi = \begin{pmatrix} 2.25 \\ 2.75 \end{pmatrix} = 2.5 \begin{pmatrix} 1 \\ 1 \end{pmatrix} + 0.25 \begin{pmatrix} -1 \\ 1 \end{pmatrix} \quad (26)$$

인데, $(-1, 1)^\top$ 이 바로 고윳값 0의 고유벡터다. 이 성분은 첫 곱셈에서 완전히 소멸하므로 $k \geq 1$ 에서는 $e_k = \gamma^k \cdot 2.5 \mathbf{1}$ 이 되어 비율이 정확히 γ 가 된다. 첫 걸음의 비율만 $2.25/2.75 = 0.8182$ ($\gamma = 0.5$ 에서는 $0.25/0.75 = 1/3$)로 더 크게 줄어드는 것이 그 성분이 사라지는 순간이다.

이 관찰은 예제 코드의 다음 한 줄을 설명해 준다.

```
# k = 0은 초깃값에서 출발하는 첫 걸음이라 예외적으로 더 많이 줄어든다
ratios = [errors[k + 1] / errors[k] for k in range(1, 11)]
assert np.allclose(ratios, gamma, atol=1e-6)
```

코드 2. 첫 걸음을 제외하고 비율을 재는 이유 (s02_bellman_iterative.py)

`range(1, 11)`로 $k = 0$ 을 건너뛴 것은 편의가 아니라 위 분해의 귀결이다. 과도 구간이 끝난 뒤 비율이 γ 로 고정된다는 사실은 γ 가 수렴 속도를 지배한다는 것, 곧 $\gamma \rightarrow 1$ 에서 반복법이 느려지는 이유를 그대로 말해 준다.

3.4 [보충] 직접 해법과 반복법의 비용

식 (18)이 정확해를 주는데도 다음 장이 반복법으로 가는 이유는 비용에 있다. 두 방법의 연산량을 부동소수점 연산 횟수로 어렵하면 다음과 같다.

- **직접 해법.** $(I - \gamma P_\pi)v = r_\pi$ 를 가우스 소거로 풀면 약 $\frac{2}{3}|S|^3$ 회. 추가로 $|S|^2$ 개의 원소를 기억해야 한다.
- **반복법.** 한 번의 갱신은 상태마다 모든 행동과 그 다음 상태를 훑으므로 약 $2|S||A|$ 회 (결정적 전이 기준). 오차를 ε 아래로 낮추는 데 필요한 반복 횟수는 식 (23)에서

$$K(\varepsilon, \gamma) = \frac{\ln(\varepsilon/\|e_0\|_\infty)}{\ln \gamma} \quad (27)$$

이므로 총 $2K|S||A|$ 회다. 기억해야 할 것은 가치 벡터 $|S|$ 개뿐이다.

두 값이 같아지는 지점이 손익분기다. $\frac{2}{3}|S|^3 = 2K|S||A|$ 에서

$$|S|^* = \sqrt{3K(\varepsilon, \gamma)|A|} \quad (28)$$

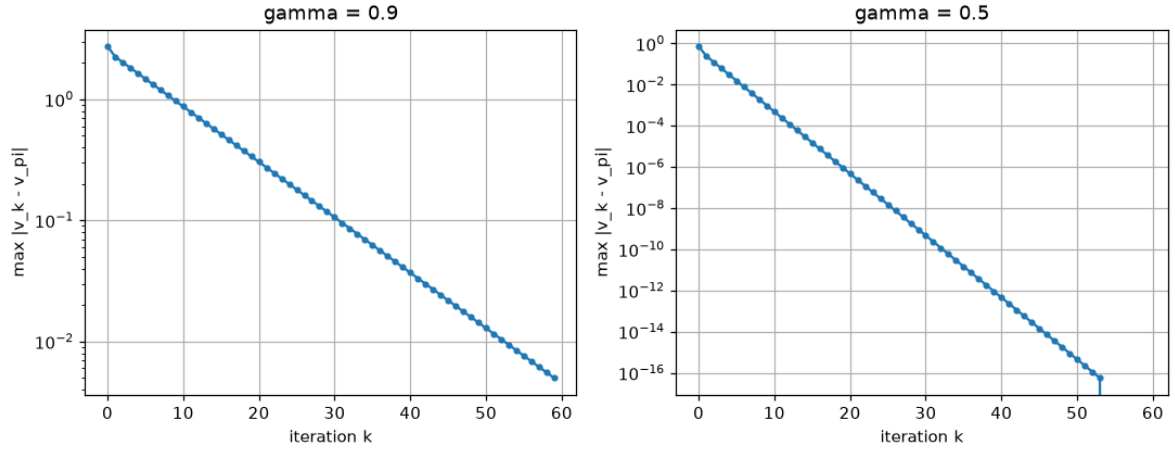


그림 8. 반복 대입의 오차(로그 눈금). 두 그래프 모두 직선인데, 로그 눈금에서의 직선은 지수적 감소를 뜻하고 그 기울기가 곧 γ 다. 왼쪽($\gamma = 0.9$)은 60회를 돌려도 오차가 4.5×10^{-3} 남지만 ($0.9^{60} \approx 1.8 \times 10^{-3}$), 오른쪽($\gamma = 0.5$)은 53회 부근에서 선이 끊긴다. 그 지점이 0.5^k 가 배정밀도 분해능 아래로 내려가 오차가 정확히 0이 된 곳이다. `s02_bellman_iterative.py`에서 생성.

표 5. 할인율에 따른 필요 반복 횟수와 손익분기 상태 수($\varepsilon = 10^{-10}$, $\|e_0\|_\infty = 1$, $|\mathcal{A}| = 4$). 마지막 열보다 상태가 많으면 반복법이 유리하다.

γ	$K(\varepsilon, \gamma)$	손익분기 $ \mathcal{S} ^*$
0.5	33.2	20.0
0.7	64.6	27.8
0.9	218.5	51.2
0.95	448.9	73.4
0.99	2291.1	165.8

를 얻는다. 표 5가 $\varepsilon = 10^{-10}$, $|\mathcal{A}| = 4$ 에서의 값이다.

$\gamma = 0.9$ 에서 손익분기는 약 51개 상태다. 본고가 다룬 두 환경은 상태가 2개와 12개이므로 모두 손익분기 아래에 있고, 실제로 `s01_bellman_linear.py`가 직접 해법을 쓴 것이 옳다. 그러나 이 우위는 $|\mathcal{S}|$ 이 세제공으로 커지는 항에 기대고 있어 금세 뒤집힌다. $|\mathcal{S}| = 1000$ 이면 직접 해법이 6.7×10^8 회인 데 비해 반복법은 1.7×10^6 회로 400배 가까이 적다.

연산량보다 먼저 걸리는 것은 기억 용량이다. 직접 해법은 $|\mathcal{S}| \times |\mathcal{A}|$ 행렬을 실체화해야 한다. $|\mathcal{S}| = 10^4$ 이면 배정밀도로 0.8GB이고, $|\mathcal{S}| = 10^6$ 이면 8TB로 어떤 기계에도 올라가지 않는다. 반면 반복법이 들고 있어야 하는 것은 길이 $|\mathcal{S}|$ 의 벡터 두 개뿐이다. 상태 공간이 커질 때 직접 해법이 먼저 무너지는 곳은 시간이 아니라 공간이다.

정리하면 두 방법의 관계는 다음과 같다. 직접 해법은 정확해를 유한 단계에 주지만 $|\mathcal{S}|$ 에 세제공으로, 기억 용량에 세제공으로 비례한다. 반복법은 근사해만 주지만 비용이 $|\mathcal{S}|$ 에 선형이며, 필요한 정확도만큼만 반복하면 된다. 다음 장의 동적 프로그래밍이 후자를 택하는 이유이고, 5장 이후 상태 공간이 더 커질 때 가치 함수를 신경망으로 근사하게 되는 출발점이기도 하다.

4 행동 가치 함수(Q 함수)와 벨만 방정식

4.1 행동 가치 함수

지금까지의 $v_\pi(s)$ 는 “상태 s 에서 정책 π 를 따를 때”의 기대 수익이다. 여기서 첫 행동만 정책의 손에서 빼앗아 자유롭게 지정한 것이 행동 가치 함수(action-value function), 흔히 Q 함수다.

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]. \quad (29)$$

식 (7)과 비교하면 조건에 $A_t = a$ 가 하나 더 붙었을 뿐이다.

그림 3-11 상태 가치 함수와 Q 함수의 백업 다이어그램

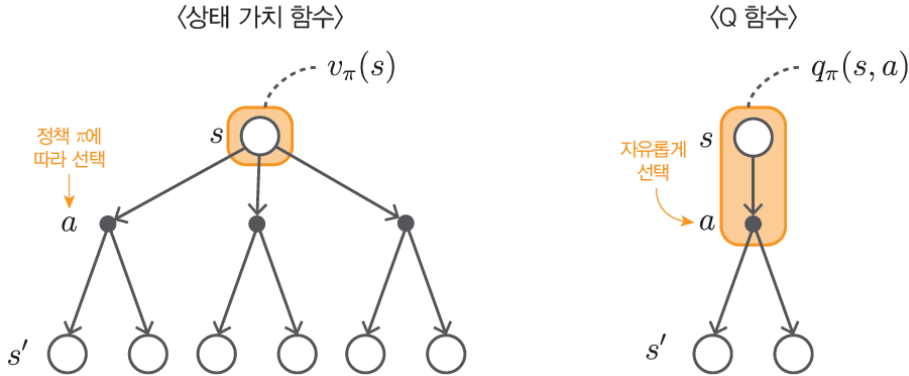


그림 9. v_π (왼쪽)와 q_π (오른쪽)의 차이. 주황색으로 감싼 범위가 왼쪽은 상태 s 만이고 오른쪽은 s 와 행동 a 까지다. 왼쪽에서는 첫 행동도 정책이 고르므로 갈래가 여럿이지만, 오른쪽에서는 첫 행동이 지정되어 갈래가 하나로 고정되고 그 아래부터 π 를 따른다. 이 “첫 행동만 자유”라는 차이가 4장 정책 개선과 6장 Q 러닝의 근거가 된다. 출처: 원서 그림 3-11.

v_π 를 유도할 때와 똑같은 순서를 밟는다. 식 (6)을 대입하고 선형성으로 나눈 뒤 두 번째 항에 $v_\pi(s')$ 를 쓰면

$$q_\pi(s, a) = \sum_{s'} p(s' \mid s, a) \{ r(s, a, s') + \gamma v_\pi(s') \} \quad (30)$$

를 얻는다. 식 (12)에서 바깥쪽 $\sum_a \pi(a \mid s)$ 만 떼어낸 모양이다. 정책이 고르던 자리를 우리가 대신 골랐으니 그 평균이 사라지는 것은 당연하다. 반대 방향은 그 평균을 도로 씌우면 된다.

$$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a). \quad (31)$$

식 (30)과 (31)은 V 와 Q 가 같은 정보의 두 표현임을 말한다. 그렇다면 왜 둘 다 필요한가. 표 6가 그 답이다.

표 6. V 와 Q 의 비교. 마지막 열이 5장 이후 모델 없는 기법이 모두 Q 를 쓰는 이유다.

	입력	첫 행동을 고르는 주체	행동을 고르려면
$v_\pi(s)$	상태	정책 π	p 와 r 를 알아야 함(식 (44))
$q_\pi(s, a)$	상태 + 행동	내가 지정	$\arg \max_a q_\pi(s, a)$ 면 끝
기억 용량	$ S $ 개	—	$ S \times A $ 개

Q 는 $|A|$ 배의 기억 용량을 쓰는 대신, 행동을 고를 때 환경 모델 p 와 r 를 참조하지 않아도 되게 해 준다. 환경 모델을 모르는 채로 경험만 가지고 배우는 5장 이후의 기법들이 예외 없이 Q 를 추정하는 것은 이 한 줄 때문이다.

4.2 행동 가치 함수를 이용한 벨만 방정식

식 (30)의 우변에는 아직 v_π 가 남아 있다. 여기에 식 (31)을 대입하면 Q 만으로 닫힌 식이 된다.

$$q_\pi(s, a) = \sum_{s'} p(s' | s, a) \left\{ r(s, a, s') + \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a') \right\}. \quad (32)$$

식 (11)과 견주면 안쪽에 $\sum_{a'} \pi(a' | s')$ 가 새로 생겼다. 다음 상태에서 고를 행동까지 정책으로 평균낸다는 뜻이다. 이 부분은 6장 SARSA의 갱신식에 그대로 나타난다.

예제 s03_q_function.py가 두 칸 세계에서 이 관계들을 수치로 확인한다. 표 7가 그 결과다.

표 7. 두 칸짜리 그리드 월드에서 무작위 정책의 Q 함수($\gamma = 0.9$). 마지막 열은 식 (31)으로 복원한 v_π 이며 표 1의 값과 일치한다.

s	$q_\pi(s, \text{Left})$	$q_\pi(s, \text{Right})$	$\frac{1}{2}(q_\pi(s, \text{L}) + q_\pi(s, \text{R})) = v_\pi(s)$
L1	-3.0250	-1.4750	-2.2500
L2	-2.0250	-3.4750	-2.7500

표 7에서 눈여겨볼 것은 $q_\pi(\text{L1}, \text{Right}) = -1.4750$ 이 $v_\pi(\text{L1}) = -2.2500$ 보다 크다는 점이다. L1에서 무작위로 고르는 것보다 Right를 고르는 편이 낫다는 뜻이며, 그 차이 0.775가 곧 개선의 여지다. L2에서도 Left(-2.0250)가 평균(-2.7500)보다 낫다. V 만 들여다볼 때는 보이지 않던 “어느 방향으로 고치면 되는가”가 Q 에서는 표를 훑는 것만으로 읽힌다.

4.3 [보충] 한 번의 탐욕화가 주는 것과 주지 못하는 것

그렇다면 Q 를 한 번 탐욕화하면 얼마나 좋아지는가. 3×4 그리드에서 무작위 정책의 q_π (표 2의 v_π 로부터 식 (30)로 얻은 것)를 탐욕화한 정책 $\pi'(s) = \arg \max_a q_\pi(s, a)$ 을 만들고, 그 정책의 가치 $v_{\pi'}$ 를 다시 연립방정식으로 정확히 풀어 비교하였다. 표 8가 결과다.

두 가지를 읽을 수 있다.

주는 것: 모든 상태에서의 개선. 개선 열에 음수가 하나도 없다. 10개 상태 전부에서 가치가 올랐고 평균은 -0.2124에서 0.7595로 바뀌었다. 이것은 우연이 아니라 정책 개선 정리의 내용이다. π' 가 q_π 에 대해 탐욕적이면 $q_\pi(s, \pi'(s)) = \max_a q_\pi(s, a) \geq \sum_a \pi(a | s) q_\pi(s, a) = v_\pi(s)$ 이고, 이를 반복 전개하면 모든 상태에서 $v_{\pi'}(s) \geq v_\pi(s)$ 가 따라온다. 최댓값은 평균보다 작을 수 없다는 사실 하나가 전부다. 무작위 정책이라는 형편없는 출발점에서 단 한 번의 탐욕화로 최적 평균 0.8263의 92%에 도달한 셈이다.

표 8. 3×4 그리드에서 한 번의 탐욕화가 주는 개선($\gamma = 0.9$). v_π 는 무작위 정책, $v_{\pi'}$ 는 이를 한 번 탐욕화한 정책, v_* 는 최적 가치다. 벽과 목표 상태는 제외하였다.

상태 s	$v_\pi(s)$	$v_{\pi'}(s)$	$v_*(s)$	개선 $v_{\pi'} - v_\pi$	남은 차이 $v_* - v_{\pi'}$
(0, 0)	0.0257	0.8100	0.8100	+0.7843	0.0000
(0, 1)	0.0946	0.9000	0.9000	+0.8054	0.0000
(0, 2)	0.2055	1.0000	1.0000	+0.7945	0.0000
(1, 0)	-0.0318	0.7290	0.7290	+0.7608	0.0000
(1, 2)	-0.4980	0.9000	0.9000	+1.3980	0.0000
(1, 3)	-0.3727	1.0000	1.0000	+1.3727	0.0000
(2, 0)	-0.1034	0.6561	0.6561	+0.7595	0.0000
(2, 1)	-0.2210	0.5905	0.7290	+0.8115	0.1385
(2, 2)	-0.4368	0.5314	0.8100	+0.9683	0.2786
(2, 3)	-0.7857	0.4783	0.7290	+1.2640	0.2507
평균	-0.2124	0.7595	0.8263	+0.9719	0.0668

주지 못하는 것: 최적성. 그러나 마지막 열에 0이 아닌 값이 셋 남아 있다. 아래쪽 세 칸 (2, 1), (2, 2), (2, 3)은 최적값에 각각 0.139, 0.279, 0.251 못 미친다. 정책을 직접 비교하면 이유가 분명해진다.

표 9. 한 번 탐욕화한 정책 π' 와 최적 정책 μ_* . 아래쪽 세 칸에서만 갈린다.

	$\pi' = \arg \max_a q_\pi(s, a)$				$\mu_* = \arg \max_a q_*(s, a)$			
	$w = 0$	$w = 1$	$w = 2$	$w = 3$	$w = 0$	$w = 1$	$w = 2$	$w = 3$
$h = 0$	RIGHT	RIGHT	RIGHT	GOAL	RIGHT	RIGHT	RIGHT	GOAL
$h = 1$	UP	WALL	UP	UP	UP	WALL	UP	UP
$h = 2$	UP	LEFT	LEFT	LEFT	UP	RIGHT	UP	LEFT

π' 는 아래쪽 줄에서 모두 왼쪽으로 돌아간다. 무작위 정책 아래에서는 오른쪽 열이 폭탄(1, 3)의 영향으로 크게 음수였고(표 2의 -0.4980, -0.7857), 그 값을 근거로 탐욕화했으니 오른쪽 열을 피하는 것이 당연하다. 그러나 그 음수는 무작위로 걸을 때 폭탄에 빨려 들어가기 때문에 생긴 값이다. 정책이 이미 개선되고 나면 (2, 2)에서 위로 올라가 (1, 2) $\rightarrow$ (0, 2) $\rightarrow$ 목표로 가는 짧은 길이 안전해진다. π' 는 낮은 가치 함수를 근거로 판단했으므로 이 사실을 알 수 없다.

여기서 다음 장의 필요성이 나온다. 탐욕화한 정책 π' 의 가치를 다시 평가하고 그 결과로 다시 탐욕화하면, 방금 안전해진 길이 값에 반영되어 정책이 또 바뀐다. 이 “평가 $\rightarrow$ 탐욕화”의 반복이 4장 정책 반복법이며, 표 8의 마지막 열에 남은 0.0668이 그 반복이 아직 회수하지 못한 몫이다. 한 번의 탐욕화는 정책 반복법의 첫 걸음일 뿐이지 그 전부가 아니다.

5 벨만 최적 방정식

지금까지는 주어진 정책 π 를 평가했다. 이제 최적 정책의 가치를 정책을 거치지 않고 직접 구한다.

5.1 상태 가치 함수의 벨만 최적 방정식

최적 정책 π_* 도 정책이므로 그 가치 함수 v_* 는 벨만 방정식을 만족한다.

$$v_*(s) = \sum_a \pi_*(a | s) \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_*(s')\}. \quad (33)$$

여기에 앞 장에서 확인한 성질, 즉 최적 정책 중에는 결정적 정책이 있다는 사실을 쓴다.

그림 3-12 세 가지 행동 중 어떤 행동을 선택할까?

$$v_*(s) = \sum_a \pi_*(a|s) \sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma v_*(s')\}$$

$= \begin{cases} -2 & (a_1) \\ 0 & (a_2) \\ 4 & (a_3) \end{cases}$

그림 10. 안쪽 합(하늘색)의 값이 행동마다 $-2, 0, 4$ 로 계산된 상황. 결정적 정책이라면 π_* 는 가장 큰 a_3 에 확률 1을 줄 수밖에 없고, 그러면 $\sum_a \pi_*(a|s)(\dots)$ 전체가 $\max_a(\dots)$ 와 같아진다. 출처: 원서 그림 3-12.

그림 10의 논리를 식으로 옮기면 벨만 최적 방정식이다.

$$v_*(s) = \max_a \sum_{s'} p(s' | s, a) \{ r(s, a, s') + \gamma v_*(s') \}. \quad (34)$$

상태 전이가 결정적이면 더 짧아진다.

$$v_*(s) = \max_a \{ r(s, a, f(s, a)) + \gamma v_*(f(s, a)) \}. \quad (35)$$

식 (11)과 식 (34)의 차이는 $\sum_a \pi(a|s)$ 자리에 $\max_a$ 가 들어간 것 하나뿐이다. 그러나 이 한 글자가 세 가지를 바꾼다.

1. **정책이 식에서 사라졌다.** 식 (34)에는 π 가 없다. 최적 정책이 무엇인지 모르는 채로 최적 가치를 구할 수 있고, 그 뒤에 정책을 뽑아내면 된다. 이것이 얻은 것이다.
2. **일차식이 아니다.** $\max$ 는 선형이 아니므로 §3처럼 행렬로 묶어 한 번에 풀 수 없다. 이것이 잃은 것이다.
3. **반복 대입은 그대로 통한다.** $\max$ 가 들어와도 축소 사상이라는 성질은 유지된다 (§5.3). 그래서 4.5절 가치 반복법이 성립한다.

5.2 행동 가치 함수의 벨만 최적 방정식

Q 에 대해서도 같은 치환이 성립한다. 식 (32)에서 다음 상태의 행동을 정책으로 평균내던 $\sum_{a'} \pi(a' | s')$ 를 $\max_{a'}$ 로 바꾸면 된다.

$$q_*(s, a) = \sum_{s'} p(s' | s, a) \left\{ r(s, a, s') + \gamma \max_{a'} q_*(s', a') \right\}. \quad (36)$$

식 (36)에서 $\max$ 가 안쪽에 있다는 점은 따로 기억해 둘 만하다. 6장 Q 러닝의 갱신식

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (37)$$

의 대괄호 안이 정확히 식 (36)의 우변에서 좌변을 뺀 것이다. 식 (32)의 SARSA 형태와 나란히 놓으면 두 알고리즘의 차이가 $\sum_{a'} \pi(a' | s')$ 대 $\max_{a'}$ 하나로 줄어든다.

5.3 [보충] 최적 벨만 연산자도 축소 사상이다

$\max$ 때문에 행렬 풀이는 막혔지만 반복 대입은 통한다고 했다. 그 근거를 확인한다. 최적 벨만 연산자를

$$(\mathcal{T}_*v)(s) = \max_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v(s')\} \quad (38)$$

로 두면, 벨만 최적 방정식은 $v_* = \mathcal{T}_*v_*$ 라는 고정점 방정식이다.

§3.2에서 본 $\mathcal{T}_\pi$ 와 견주면 달라진 곳은 하나뿐이다. 둘 다 표 v 를 받아 표를 내놓는 기계이고 둘 다 “한 걸음은 진짜로 굴리고 나머지는 v 로 때운다”는 계산을 하지만, $\mathcal{T}_\pi$ 는 행동마다 나온 값을 정책 확률로 평균내는 데 비해 $\mathcal{T}_*$ 는 그중 가장 큰 것을 고른다. 상태마다 모든 행동을 하나씩 시켜 보고 성적이 제일 좋은 것을 그 칸에 적는 기계라고 읽으면 된다.

두 칸 세계에서 이 기계를 손으로 돌려 보면 하는 일이 그대로 드러난다. 상태 전이가 결정적이므로 식 (35)에 따라 상태마다 후보가 둘뿐이다. 표 10가 $v_0 = 0$ 에서 출발한 처음 여섯 번이다.

표 10. 두 칸 세계에서 $v_{k+1} = \mathcal{T}_*v_k$ 를 손으로 돌린 것($\gamma = 0.9$). 가운데 두 묶음은 그 칸에서 계산되는 두 후보 $r + \gamma v_k(s')$ 이고, 굵은 값이 $\max$ 로 뽑혀 다음 줄의 v_{k+1} 이 된다. 굵은 자리가 한 번도 바뀌지 않는 것과, 맨 아랫줄에서 넣은 값과 뽑힌 값이 같아지는 것이 볼 곳이다.

	현재 표 v_k		L1의 두 후보		L2의 두 후보	
k	$v_k(\text{L1})$	$v_k(\text{L2})$	Left	Right	Left	Right
0	0.0000	0.0000	-1.0000	1.0000	0.0000	-1.0000
1	1.0000	0.0000	-0.1000	1.0000	0.9000	-1.0000
2	1.0000	0.9000	-0.1000	1.8100	0.9000	-0.1900
3	1.8100	0.9000	0.6290	1.8100	1.6290	-0.1900
4	1.8100	1.6290	0.6290	2.4661	1.6290	0.4661
5	2.4661	1.6290	1.2195	2.4661	2.2195	0.4661
∞	5.2632	4.7368	3.7368	5.2632	4.7368	3.2632

표 10가 보여주는 것이 둘이다. 첫째, 굵은 자리가 $k = 0$ 부터 줄곧 (Right, Left)이며 한 번도 옮겨 다니지 않는다. §6.3에서 손풀이가 가정하고 검증해야 했던 그 선택을 $\max$ 가 매 반복 스스로 고르고 있는 것이다. 둘째, 맨 아랫줄에서 넣은 표 (5.2632, 4.7368)과 뽑혀 나온 표가 정확히 같다. 넣은 것이 그대로 나왔으므로 이 표가 $\mathcal{T}_*$ 의 고정점이고, 그것이 곧 v_* 다.

남은 문제는 이 반복이 항상 그 고정점으로 가느냐다. 핵심은 $\max$ 가 비확장적이라는 다음 보조 사실이다.

명제 2 ($\max$ 의 비확장성). 임의의 유한 집합 $\mathcal{A}$ 위의 함수 f, g 에 대해 $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$ 이다.

증명. $\bar{a} = \arg \max_a f(a)$ 라 하면 $\max_a f(a) - \max_a g(a) \leq f(\bar{a}) - g(\bar{a}) \leq \max_a |f(a) - g(a)|$ 이다. f 와 g 의 역할을 바꾸면 반대 방향의 부등식도 같은 방식으로 얻어진다. $\square$

따름정리 3. $\|\mathcal{T}_*v - \mathcal{T}_*w\|_\infty \leq \gamma \|v - w\|_\infty$ 이며, 따라서 $\mathcal{T}_*$ 는 γ -축소 사상이다. 벨만 최적 방정식의 해는 유일하게 존재하고, 임의의 v_0 에서 출발한 $v_{k+1} = \mathcal{T}_*v_k$ 는 v_* 로 수렴한다.

증명. 각 s 에 대해 명제 2를 적용하면 $|(\mathcal{T}_*v)(s) - (\mathcal{T}_*w)(s)| \leq \max_a |\gamma \sum_{s'} p(s' | s, a) \{v(s') - w(s')\}| \leq \gamma \|v - w\|_\infty$ 이다. 나머지는 바나흐 고정점 정리다. $\square$

정리하면 $\max$ 는 선형성만 깨뜨렸을 뿐 축소성은 건드리지 않았다. 명제 1에서 γ 가 나온 자리도 $\max$ 의 비확장성 뒤에 그대로 남아 있으므로, §3.3의 수렴률 논의와 §3.4의 반복 횟수 추정이 최적 방정식에도 적용된다. 4장의 정책 평가와 가치 반복법이 사실상 같은 알고리즘인 이유다.

6 벨만 최적 방정식의 예

6.1 벨만 최적 방정식 적용

같은 두 칸짜리 그리드 월드로 돌아간다. 이번에는 정책이 주어지지 않는다. 최적 정책이 무엇인지도 함께 찾아야 한다.

그림 3-13 두 칸짜리 그리드 월드

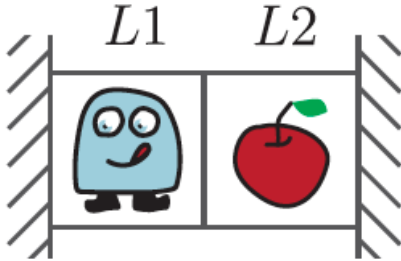


그림 3-14 상태 L1과 L2를 시작점으로 한 백업 다이어그램

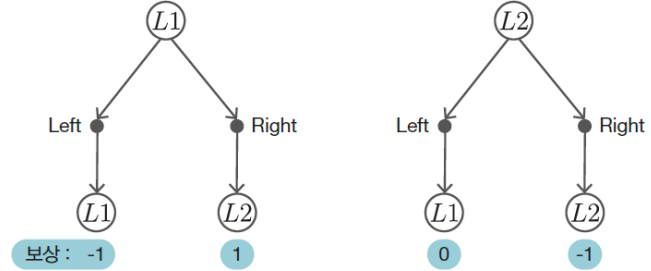


그림 11. 정책이 주어지지 않은 두 칸 세계(왼쪽)와 L1, L2에서 각각 시작하는 한 단계짜리 다이어그램(오른쪽). 그림 6 왼쪽에 있던 막대그래프가 사라졌고, 그림 7과 달리 가지에 확률 0.5가 붙어 있지 않다. 확률로 평균내는 대신 둘 중 큰 쪽을 고른다. 출처: 원서 그림 3-13, 3-14.

그림 11 오른쪽을 읽으면 두 식이 나온다.

$$\begin{cases} v_*(L1) = \max\{-1 + 0.9 v_*(L1), 1 + 0.9 v_*(L2)\}, \\ v_*(L2) = \max\{0 + 0.9 v_*(L1), -1 + 0.9 v_*(L2)\}. \end{cases} \quad (39)$$

max가 있는 채로는 풀 수 없으므로, 어느 쪽이 이길지 가정하고 풀어본 뒤 검증한다. L1에서 Right, L2에서 Left가 이긴다고 가정하면 max가 사라져 일차식이 된다.

$$v_*(L1) = 1 + \gamma v_*(L2), \quad v_*(L2) = 0 + \gamma v_*(L1). \quad (40)$$

두 번째 식을 첫 번째에 대입하면

$$v_*(L1) = \frac{1}{1 - \gamma^2} = 5.2632, \quad v_*(L2) = \frac{\gamma}{1 - \gamma^2} = 4.7368 \quad (41)$$

이다. 이제 가정을 검증한다. 이 값을 식 (39)의 우변에 넣으면

$$\begin{aligned} L1: & -1 + 0.9 \times 5.2632 = 3.7368 < 1 + 0.9 \times 4.7368 = 5.2632 \quad \Rightarrow \text{Right} \checkmark \\ L2: & 0 + 0.9 \times 5.2632 = 4.7368 > -1 + 0.9 \times 4.7368 = 3.2632 \quad \Rightarrow \text{Left} \checkmark \end{aligned}$$

로 가정한 행동이 실제로 max를 달성한다. 따라서 식 (41)이 벨만 최적 방정식의 해다.

예제 s04_bellman_optimal.py는 손풀이를 거치지 않고 식 (35)을 그대로 반복 대입하여 같은 값에 도달한다. max가 매 반복마다 스스로 옳은 가치를 골라 주므로 가정과 검증이라는 절차 자체가 필요 없다.

예제는 왜 정확히 333회에서 멈추는가. 이 예제는 갱신량 delta가 정확히 0이 될 때까지 돌아 333회에서 멈춘다. 이 숫자는 부동소수점 분해능에서 나온다. 수렴값 $v_*(L1) = 5.2632$ 부근에서 배정밀도의 최소 간격(1 ulp)은 8.882×10^{-16} 인데, 이론적 갱신량 γ^k 가 이 값 아래로 내려가는 지점이

$$k = \frac{\ln(8.882 \times 10^{-16})}{\ln 0.9} = 328.9 \quad (42)$$

그림 3-16 두 칸짜리 그리드 월드의 최적 정책

그림 3-15 백업 다이어그램과 최적 상태 가치 함수

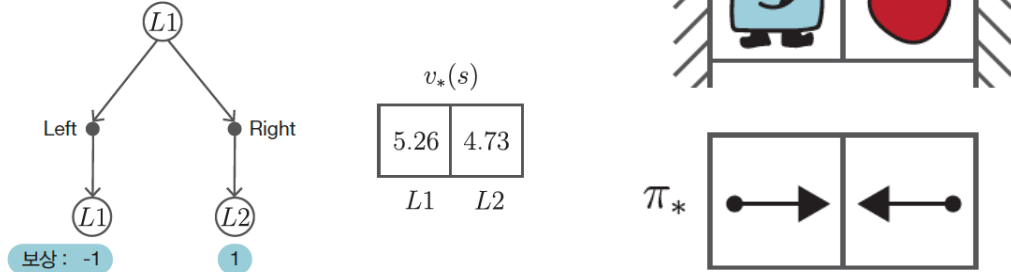


그림 12. 최적 상태 가치 함수(왼쪽)와 최적 정책(오른쪽). 앞 장에서 네 가지 정책을 전부 평가해 얻었던 값 및 정책 μ_2 와 같다. 이번에는 정책을 하나도 나열하지 않고 방정식만으로 얻었다. 출처: 원서 그림 3-15, 3-16.

이다. 실측하면 $k = 328$ 부터 δ 가 정확히 1 ulp에 고정된 채 다섯 번 반복되다가 333회에서 0이 된다. 즉 알고리즘이 수렴을 판정한 것이 아니라 더 이상 표현할 수 있는 다음 값이 없어 멈춘 것이다. 실용적으로는 이렇게 0을 기다리기보다 문턱 ϵ 을 두고 식 (27)만큼만 도는 편이 낫다.

같은 반복을 3×4 그리드에 적용하면 6회 만에 끝난다. 두 칸 세계와 달리 목표 상태라는 흡수 상태가 있어 가치가 유한 걸음 안에 확정되기 때문이며, 반복 횟수를 지배하는 것이 γ 만은 아님을 보여준다.

6.2 최적 정책 구하기

최적 가치를 알면 최적 정책은 탐욕적으로 뽑는다. Q 가 있으면 한 줄이다.

$$\mu_*(s) = \arg \max_a q_*(s, a). \quad (43)$$

V 만 있으면 한 걸음 내다봐야 한다.

$$\mu_*(s) = \arg \max_a \sum_{s'} p(s' | s, a) \{ r(s, a, s') + \gamma v_*(s') \}. \quad (44)$$

식 (44)의 우변에 p 와 r 가 등장한다는 것이 표 6가 말한 Q 의 장점이 드러나는 자리다. 환경 모델을 모르면 V 에서 정책을 뽑을 수 없다.

두 칸 세계에서 식 (44)를 적용하면 L1에서 Right, L2에서 Left가 나온다(그림 12 오른쪽). 앞 장이 네 가지 정책을 전부 평가해 골라냈던 μ_2 와 같은 정책이며, 사과를 먹고 되돌아오기를 반복하는 행동이다.

6.3 [보충] 네 가지 가정 중 무엇이 자기일관인가

§6.1의 손품이는 “L1에서 Right, L2에서 Left”라는 가정에서 출발했다. 이 가정이 맞았음을 사후 검증했지만, 다른 세 가정을 택했다면 어떤 일이 벌어졌을지는 확인하지 않았다. 결정적 정책이 네 가지뿐이므로 전부 풀어 볼 수 있다. 가정한 정책 μ 에 대해 $\max$ 를 제거한 연립 일차방정식 $(I - \gamma P_\mu)v = r_\mu$ 를 풀고, 그 해가 식 (39)의 $\max$ 를 실제로 달성하는지 검사한 결과가 표 11다.

표 11는 손품이의 논리 구조를 드러낸다. 벨만 최적 방정식을 푸는 것은 $|A|^{|S|}$ 개의 경우 각각에 대해 선형 방정식을 풀고 자기일관성을 검사하는 일과 같으며, 그중 정확히 하나가 통과한다. 경우의

표 11. 네 가지 결정적 정책 가정에 대한 해와 자기일관성 검사($\gamma = 0.9$). “달성 행동” 열은 그 해를 식 (39)의 우변에 넣었을 때 실제로 $\max$ 를 주는 행동이다. 가정과 일치하는 것은 한 행뿐이다.

가정 ($\mu(L1), \mu(L2)$)	$v(L1)$	$v(L2)$	달성 행동	자기일관
(Left, Left)	-10.0000	-9.0000	(Right, Left)	모순
(Left, Right)	-10.0000	-10.0000	(Right, Left)	모순
(Right, Left)	5.2632	4.7368	(Right, Left)	일관
(Right, Right)	-8.0000	-10.0000	(Right, Left)	모순

수는 전수 탐색과 같지만 각 경우의 비용이 무한 궤적 전개에서 선형 방정식 풀이로 줄었다는 것이 벨만 방정식이 준 것이다. 그리고 반복 대입은 이 경우 분석을 아예 건너뛴다. $\max$ 가 매 반복마다 스스로 옳은 가치를 골라 주기 때문이다.

세 모순 행의 값들도 읽어 둘 만하다. -10 은 $-1/(1 - \gamma)$ 로, 영원히 벽에 부딪히며 -1 을 받는 정책의 가치다. (Left, Right)는 두 상태 모두 각자의 벽에 갇히는 정책이라 양쪽 다 -10 이 된다. 가정이 나쁘면 방정식은 여전히 답을 주되 그 답은 “그 나쁜 정책의 가치”일 뿐이며, 자기일관성 검사만이 그것을 걸러낸다. 검사를 생략하면 $\max$ 를 임의로 제거한 식의 해를 최적 가치로 착각하게 된다.

6.4 세 방법의 대조

예제 `s04_bellman_optimal.py`는 같은 답에 이르는 세 가지 경로를 나란히 놓고 `assert`로 일치를 확인한다. 표 12가 그 대조다.

표 12. 두 칸 세계의 최적 가치를 구하는 세 방법($\gamma = 0.9$). 세 값이 모두 일치한다.

방법	$v_*(L1)$	$v_*(L2)$	비용
반복 대입 ($v_{k+1} = \mathcal{T}_* v_k$)	5.2632	4.7368	333회 $\times S A $
순풀이 (식 (41))	5.2632	4.7368	가정 + 검증
전수 탐색 (앞 장)	5.2632	4.7368	$ A ^{ S }$ 개 정책 $\times$ 500걸음

값이 같다는 사실보다 중요한 것은 비용 열이다. 전수 탐색은 정책 수가 $|A|^{|S|}$ 로 폭발하고 각 정책마다 궤적을 500걸음 펼쳐야 한다. 벨만 최적 방정식의 반복 대입은 상태 수에 비례하는 계산을 정해진 횟수만큼 되풀이할 뿐이다. 상태가 두 개일 때는 둘 다 순식간이지만, 상태가 100개가 되면 한쪽은 4^{100} 이고 다른 쪽은 $100 \times 4 \times K$ 다.

7 정리

본고는 벨만 방정식이 앞 장에서 만난 두 벽을 어떻게 허무는지를 정리하였다.

출발점은 수익의 재귀 구조 $G_t = R_t + \gamma G_{t+1}$ 한 줄이었다(식 (6)). 이를 가치 함수의 정의에 대입하고 정책과 상태 전이라는 두 층의 확률로 전개하면 벨만 방정식 (식 (11))을 얻는다. 이 식은 무한한 미래를 “한 걸음 보상 + $\gamma \times$ 이웃 상태의 가치”로 접어 넣으므로, 가치 함수는 미지수 $|S|$ 개짜리 연립일차방정식 $(I - \gamma P_\pi)v = r_\pi$ 의 해가 된다(식 (18)). 이 방정식이 항상 풀리는 이유는 $\gamma < 1$ 이 $\rho(\gamma P_\pi) < 1$ 을 보장하기 때문이며, 그 역행렬의 노이만 급수(식 (19))는 원래의 무한급수를 행렬로 다시 쓴 것에 지나지 않는다. 앞 장이 500걸음 시뮬레이션으로 얻었던 5.2632가 2×2 연립방정식 한 번으로 나온 것이 그 요약이다.

행동 가치 함수 $q_\pi(s, a)$ 는 첫 행동만 정책에서 떼어낸 것으로, V 와는 식 (30), (31)으로 자유롭게 오간다. $|A|$ 배의 기억 용량을 쓰는 대신 환경 모델 없이 $\arg \max_a q(s, a)$ 로 행동을 고를 수 있다는 것이 Q 의 값어치이며, 5장 이후 모델 없는 기법들이 예외 없이 Q 를 추정하는 근거다. 벨만 최적 방정식은 $\sum_a \pi(a | s)$ 자리를 $\max_a$ 로 바꾼 것으로 (식 (34), (36)), 정책이 식에서 사라지는 대신

선형성을 잃는다. 최적 가치를 알고 나면 최적 정책은 $\mu_*(s) = \arg \max_a q_*(s, a)$ 로 탐욕적으로 뽑으면 되고, 두 칸 세계에서 그 답은 $v_*(L1) = 5.2632$, $v_*(L2) = 4.7368$, $\pi_* = (\text{Right}, \text{Left})$ 다. 앞 장이 정책 네 가지를 전부 평가해 얻은 것을 이 장은 방정식 두 줄로 얻었다.

보충 분석에서 얻은 결론은 다음 네 가지다.

첫째, 반복 대입의 점근 수렴률은 γ 이하가 아니라 정확히 γ 다. P_π 가 행 확률 행렬이라 $\rho(P_\pi) = 1$ 이고, 초기 오차의 1 방향 성분이 γ^k 로만 줄기 때문이다. 두 칸 세계에서는 $e_0 = 2.5 \cdot 1 + 0.25 \cdot (-1, 1)^\top$ 이며 두 번째 성분이 $\lambda = 0$ 의 고유벡터라 한 번에 소멸하므로, 첫 걸음의 비율만 0.818로 어긋나고 이후는 0.9000으로 고정된다.

둘째, 직접 해법과 반복법의 손익분기는 $|S| \approx \sqrt{3K|\mathcal{A}|}$ 이며 $\gamma = 0.9$, $\varepsilon = 10^{-10}$ 에서 약 51개 상태다. 본고의 두 환경(상태 2개, 12개)은 손익분기 아래여서 직접 해법이 적절했지만, $|S| = 10^6$ 에서 $|S| \times |S|$ 행렬은 8 TB로 기억 용량이 먼저 무너진다. 다음 장이 반복법을 택하는 이유가 여기에 있다.

셋째, 무작위 정책의 Q 를 한 번 탐욕화하면 3×4 그리드의 10개 상태 전부에서 가치가 오른다 (평균 $-0.212 \rightarrow 0.760$). 최댓값은 평균보다 작을 수 없다는 사실 하나에서 나오는 정책 개선 정리의 내용이다. 그러나 아래쪽 세 칸은 최적값에 최대 0.279 못 미치는데, 탐욕화의 근거가 된 Q 가 여전히 낡은 정책의 것이기 때문이다. 이 간극이 4장 정책 반복법이 “반복”인 이유다.

넷째, $\max$ 는 비확장적이므로 $\mathcal{T}_*$ 역시 γ -축소 사상이고, 그래서 경우 분석을 건너뛴 반복 대입이 통한다. 손풀이 쪽에서 보면 네 가지 결정적 정책 가정 중 자기일관인 것은 (Right, Left) 하나뿐이며, 나머지 셋은 여전히 해를 주되 그것은 그 나쁜 정책의 가치일 뿐이다. 예제가 333회에서 멈추는 것은 수렴 판정이 아니라 γ^k 가 $1 \text{ ulp} = 8.882 \times 10^{-16}$ 아래로 내려간 결과다.

다음 장에서는 이 방정식들을 반복 계산으로 푸는 동적 프로그래밍을 다룬다. 벨만 방정식의 반복이 정책 평가이고 벨만 최적 방정식의 반복이 가치 반복법이며, 표 8에 남은 0.0668을 회수하기 위해 평가와 탐욕화를 번갈아 돌리는 것이 정책 반복법이다.

A 재현 정보

본고의 모든 수치는 Python 3.12.13, NumPy 2.5.2, Matplotlib 3.11.1 환경에서 재현하였다. 예제는 난수를 쓰지 않으므로 시드 고정 없이 항상 같은 값이 나온다.

표 13. 예제 코드 목록. 원서 3장에는 소스 코드가 없으므로 모두 본고의 보충 예제다.

파일	내용	대응 절
ch03/s01_bellman_linear.py	$(I - \gamma P_\pi)v = r_\pi$ 의 직접 해	§3.1
ch03/s02_bellman_iterative.py	반복 대입과 오차의 지수적 감소	§3.3
ch03/s03_q_function.py	$V \leftrightarrow Q$ 변환과 식 (32) 검증	§4
ch03/s04_bellman_optimal.py	벨만 최적 방정식의 반복 대입, 손풀이, 전수 탐색 대조	§6

표 1, 2, 7, 12과 그림 8은 표 13의 예제를 그대로 실행해 얻은 출력이다. 표 4(오차와 비율의 자릿수 확장), 표 5, 표 8, 표 9, 표 11, 그리고 §6.1의 ulp 계산은 본고를 위해 별도로 계산한 값이다.

표 8의 $v_{\pi'}$ 는 탐욕화한 정책을 다시 식 (18)으로 정확히 푼 값이며, v_* 는 같은 환경에 식 (35)의 반복 대입을 수렴할 때까지(6회) 적용해 얻었다. 표 11는 네 가지 결정적 정책 각각에 대해 $(I - \gamma P_\mu)v = r_\mu$ 를 풀고 그 해를 식 (39)의 우변에 대입하여 $\arg \max$ 를 확인한 결과다.

3×4 그리드의 상태는 원서와 예제 코드의 표기를 따라 (h, w) 로 적었다. h 가 위에서부터 센 행, w 가 왼쪽에서부터 센 열이며 (0, 3)이 목표, (1, 1)이 벽, (1, 3)이 폭탄이다.

참고문헌

- [1] 사이토 고키, 밑바닥부터 시작하는 딥러닝 4: 파이썬으로 구현하는 강화 학습 알고리즘, 한빛 미디어, 2023.

- [2] sajacaros, 마르코프 결정 과정과 최적 정책: 상태가 개입할 때 무엇이 달라지는가, 2026.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [4] R. Bellman, *Dynamic Programming*, Princeton University Press, 1957.
- [5] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, Wiley, 1994.

본 문서에 실린 그림 중 “원서 그림”으로 표기한 것은 참고문헌 [1]의 도판이며, 학습 정리 목적으로 인용하였다. 그 밖의 그림과 표는 본고에서 직접 생성한 것이다.